

Line Following Robot Project Document

Group 32:

Callum Crilly, Curtis Christian, Oliver Glossop and Daniel Middleton

Team Members

Name:	Role:	Signature:
Curtis Christian	Lead Product Designer	<i>Curtis Christian</i>
Oliver Glossop	Programmer	<i>Oliver Glossop</i>
Callum Crilly	Mechanical Technician	<i>Callum Crilly</i>
Daniel Middleton	Electrical Technician	<i>Daniel Middleton</i>

Abstract

This project used a mixture of electrical, mechanical and software design to construct a mostly functional Line Following Robot featuring specially designed PCBs for sensor and motor control, powered by PID program ran on an ATmega328P-U. The physical design used two independent back wheels featuring a geared transmission from associated motors, and a third omnidirectional marble wheel to allow for smooth turning. However, this project faced continuous hardware problems throughout construction and testing which eventually led to unrecoverable damage to the PCBs and components.

While the final robot never managed to complete a full lap of the test course due to technical issues, the underlying programming and physical systems demonstrating partial functionality. The partial functionality it was able to achieve was applied use of the line sensors, a calibration routine, and proportional and differential control performing reliably during testing. While flawed, the LFR could have been made to function with only a few tweaks if the hardware instability had not caused such major circuit damage.

Despite all the setbacks this project provided valuable lessons in PCB design, embedded system programming and design iteration in a team context. Many lessons were learned which will undoubtedly lead to more successful projects in the future.

Project Overview

The directive for this project was to design and construct a Line Following Robot capable of completing an unknown course as fast as possible, racing against other LFRs. Speed was the only important design criteria for this project; a mixture of Programming, Electronic Design and Mechanical Design was required to make an efficient and powerful LFR that was still reliable. This project was constrained by a six-week deadline and a limited pool of resources, which incentivised a quick and simple solution that could feasibly be built and delivered on time.

PCB Design

The main-board architecture was designed for robustness, ease of assembly, and straightforward sensor-to-actuator control. An ATmega328P-U through-hole package was selected as the microcontroller because it simplifies soldering and allows simple replacement via a socket if the MCU is damaged. The MCU's on-chip ADC channels were used to read the line sensors directly, reducing parts count and latency, while two hardware PWM outputs control the motors for reliable speed regulation. This combination yields a compact, serviceable board that supports the robot's core functions, sensor sampling, control logic, and motor actuation, without unnecessary complexity. see Fig. 8 for full PCB layout and component application.

Power management and brownout prevention were central design themes. A dedicated 3.3 V linear regulator supplies the MCU and sensors to ensure stable logic levels and compatibility with the chosen peripheral components. Local decoupling follows best practice: 100 nF ceramic capacitors near each VCC pin and across the regulator pins to suppress high-frequency noise, complemented by electrolytic bulk capacitors to smooth low-frequency battery ripple. A 220 μ F electrolytic placed at the battery connector serves as the primary energy reservoir to reduce supply droop during motor transients; closer to the regulator we used a 22 μ F (battery side) and a 10 μ F (MCU side) electrolytic to meet the regulator's stability requirements. A user-accessible power switch was included upstream of the regulator both as a hard power-off and to protect the regulator during programming with external development hardware; a blue LED with a series resistor provides a clear visual indication of the regulator being enabled. These choices prioritise ease of use during real-world operation where motors create large, rapid current demands. See Fig. 1 below for the full voltage regulator schematic.

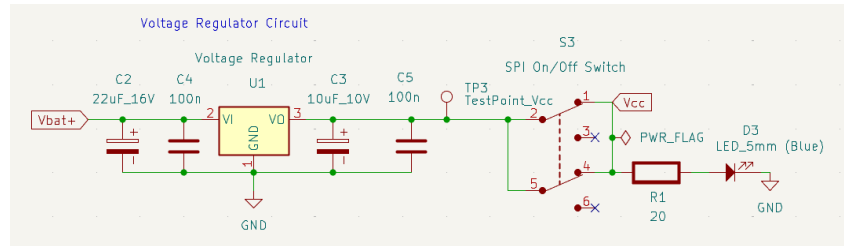


Fig. 1. Voltage regulator circuit schematic.

Motor drive circuitry was designed for simplicity and protection. Each Multicomp MM28 motor is switched on its low side by N-channel MOSFETs (chosen for low Drain-Source resistance and compact size), with the MOSFET drains tied to the motor negative and sources to the ground plane. The MOSFET gates are driven directly from MCU PWM pins, enabling efficient pulse-width speed control. Schottky diodes are placed across the motor terminals to provide a safe discharge path for inductive energy when switches open; this flyback path prevents inductive voltage spikes from damaging the MOSFETs or the MCU's I/O. A design decision to keep the battery and motor connectors as simple two-pin headers facilitates mechanical integration and quick replacement during testing. Choice of motor was also obvious, as recommended voltage to run ATmega328P-U was 3.3 V. This meant at least 3 AA NiMH batteries providing 1.2V per cell were needed. This finally meant that given the total pre-regulated voltage would be above 3V and because of this the MM28's were needed over the MM10's (they are only recommended to operate at below 3V). See Fig. 2 below, for motor driver circuit schematic.

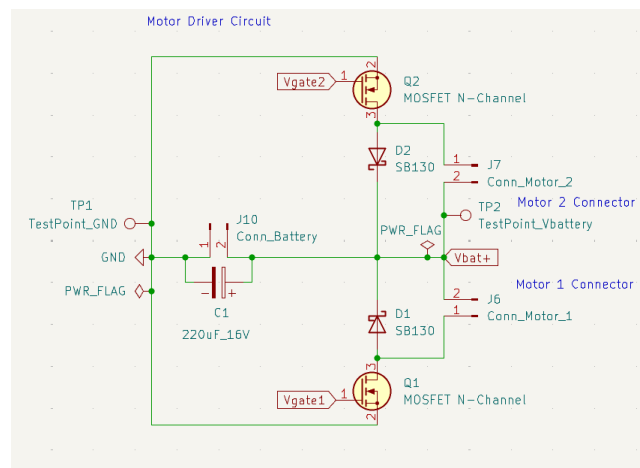
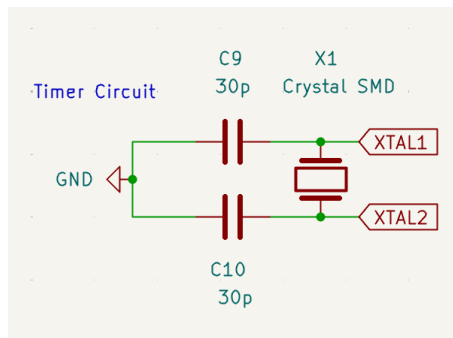


Fig. 2. Motor driver circuit schematic.

For the clock the values of external capacitance were 30pf calculated from the information in design considerations datasheet. Using the load capacitance of the (9.216 MHz) external crystal oscillator and the stray capacitance (estimated at 5 – 10 pf) gives a range of 26 – 31 pf. 30pf is the most readily available value of ceramic capacitance. See Fig. 3 below for external crystal oscillator schematic.



For programming via SPI or Serial Peripheral Interface, there is a six-pin header with recommended resistors between the SCLK, MOSI and MISO inputs there MCU pins. The resistor values are 330 ohms. These are the recommended values used to limit the current on a 3.3V board to 10mA as outlined in the design considerations data sheet. If VCC is connected from this port in programming the switch must be set to off as previously mentioned. see Fig. 4 below for the schematic of the SPI port circuit.

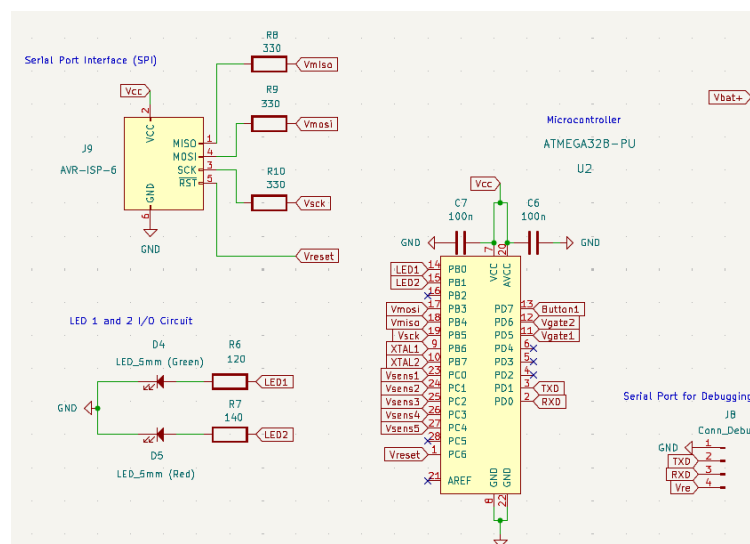


Fig. 4. Schematic of MCU circuit, serial port, SPI circuit and output LED circuit.

Component-level choices were made to balance performance, availability, and ease of debugging. Two LEDs used for status were sized for ~ 10 mA nominal forward current; calculated series resistances give predictable, visible illumination without overstressing the 3.3 V regulator or the MCU, see Fig. 4. above for these circuits. Pull-ups and pull-downs use 10 k Ω values to ensure defined logic states without large standby currents; the reset circuit uses a 10 k Ω pull-up with a momentary push button to ground for active-low reset behaviour. Similar to this is the input button used to operate states in software. The button input has the reverse configuration of the reset button, see Fig. 6 for the schematic of the circuit. The DTR–reset 100 nF capacitor allows for a momentary reset pulse instead of holding the MCU in reset for a prolonged period. See Fig. 5 below for the schematic of the full reset circuit.

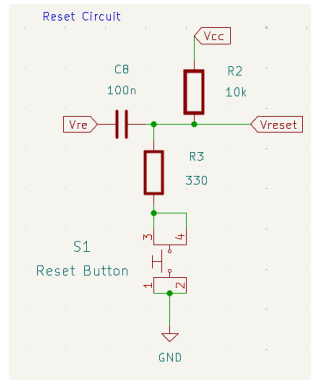


Fig. 5. Schematic of reset circuit.

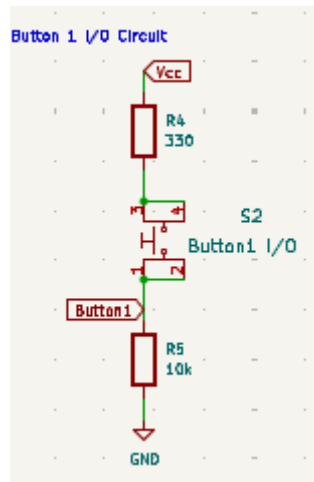


Fig. 6. Schematic of the Button input circuit.

The sensor sub-board uses a bottom mounted TCRT5000 reflectance sensors wired for analogue measurement. Each sensor's collector/emitter output feeds an ADC channel to one of the MCU's ADC pins (PC0–PC4). Each sensor's VCC is decoupled with a 100 nF capacitor and uses a modest series resistor on the photodiode LED to limit current; the collector output is pulled toward ground through a 3.3 k Ω resistor to form a stable voltage divider, allowing the ADC to read a continuous reflectance value rather than a binary threshold. See Fig. 7 and Fig. 8 for the TCRT5000 circuit schematic and PCB layout. Locating the five 3-pin headers on the board edge simplifies mechanical mounting and avoids potential component damage. The sensor vias intentionally cut the ground plane to go beneath VCC; these are the only cuts in the ground plane from vias, as good practice. See Fig. 8 to see PCB layout and full circuit combination.

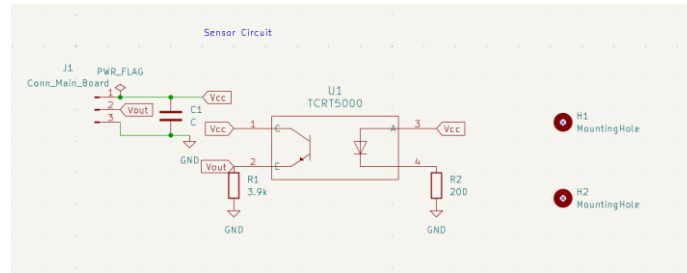


Fig. 7. Circuit schematic of TCRT5000 sensor board.

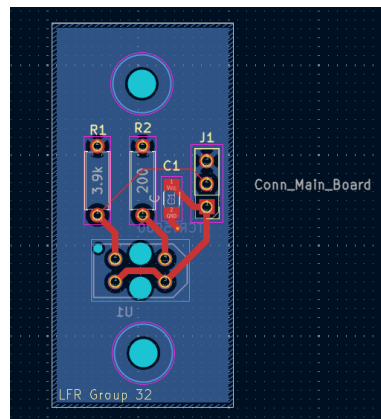


Fig. 8. PCB layout of sensor boards.

PCB layout choices were driven by noise control serviceability and simplicity. Decoupling capacitors are placed as close as physically possible to the MCU and regulator pins. The large 220 μF bulk capacitor sits adjacent to the battery connector to decouple the largest transients at the entry point. Signal traces for analogue ADC inputs are routed away from high-current motor traces and MOSFET switching nodes to minimize interference. See below for PCB layout of the main board (Fig. 9).

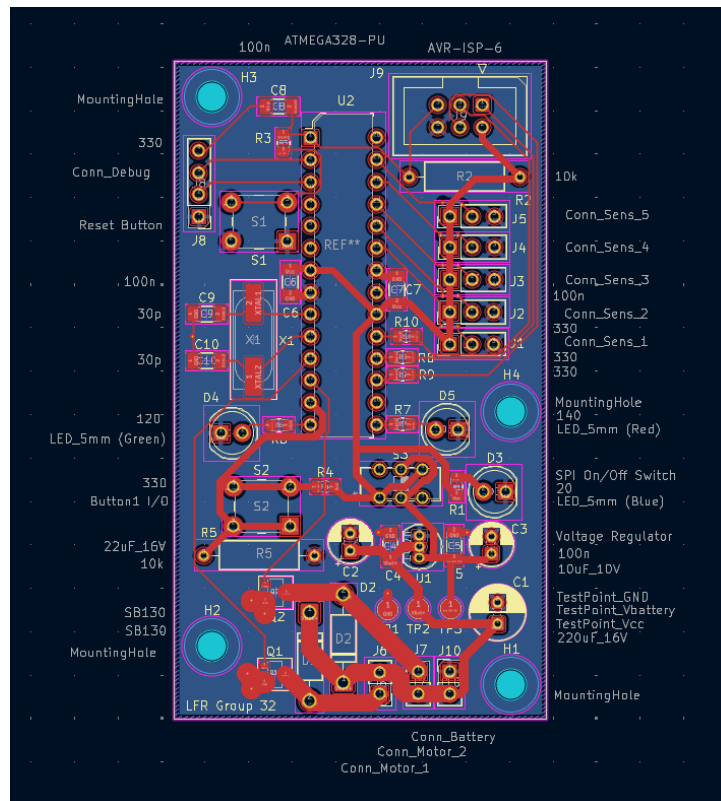


Fig. 9. PCB layout of main board.

Two minor modifications were implemented during prototyping to improve performance and usability. The blue led was calculated without luminosity intensity in mind where the green and the red were relative in terms luminosity intensity (red ~50mcd and green ~60mcd), the blue LED calculated for a forward current of 10mA (resistance of 20 Ω) had a luminosity intensity of ~300mcd. This was increased to 3.3 k Ω , reducing luminosity to an appropriate level (~10 mcd) for a passive power indicator while extending LED lifespan. Additionally, the power switch (JS202011CQN) had an incorrect pinout relative to the schematic; one leg of the switch was carefully removed to restore correct functionality without compromising the switch operation. This results in the final completed physical main PCB see below, Fig. 10.



Fig. 10. Fully soldered main PCB

Software

The control software for the line-following robot is designed as a three-stage state machine which the user progresses through using a button connected to PD7. Upon powering up, the robot enters an idle state in which no sensors are read, and no motors are driven. This initial state exists purely for safety and allows the robot to remain inactive until the user is ready to begin calibration. See Fig. 11 below for code implementation of the main loop and idle state.

```
// Loop
void loop() {
  switch (robotState) {
    case IDLE:
      idleState();
      break;
    case CALIBRATING:
      calibrateState();
      break;
    case RUNNING:
      runningState();
      break;
  }
}

// IDLE STATE
void idleState() {
  digitalWrite(ledCalibrate, LOW); // both LED's off
  digitalWrite(ledRunning, LOW);
  analogWrite(leftMotorPWM, 0); // set motors to off
  analogWrite(rightMotorPWM, 0);

  // Wait for button press to start calibration
  if (digitalRead(buttonPin) == HIGH) {
    while (digitalRead(buttonPin) == HIGH) {
      continue; // if button is held down
    }
    delay(200); // debounce
    robotState = CALIBRATING; // shift to calibration state
  }
}
```

Fig. 11. Main loop and idle state function.

When the button is pressed, the robot transitions to the calibration phase and turns on the red LED to indicate this mode to the operator. During this phase, the software continuously reads the analogue outputs from the five reflectance sensors, which are connected to the microcontroller's ADC pins PC0 to PC4. These readings are used to dynamically determine the minimum and maximum values that each sensor can produce under the current environmental lighting and track conditions. As the operator slowly moves the robot horizontally across the track ensuring each sensor passes over both the white background and the black line the software records the lowest and highest values observed for each sensor, with a small margin applied to both to ensure robustness. This adaptive calibration is essential because the sensors' behaviour can vary noticeably depending on sensor tolerance, temperature, ambient light, and the reflectivity of the track material. See Fig. 12 below for code implementation of the calibration function.

```
void calibrateState() {
    digitalWrite(ledCalibrate, HIGH); // Red LED on meaning calibration move active

    while (digitalRead(buttonPin) == LOW) { // Calibrate till button press
        for (uint8_t i = 0; i < 5; i++) {
            int16_t value = analogRead(sensorPin[i]); // Read 10 bit sensor value
            if ((value < sensorMin[i]) && (value != 0)) sensorMin[i] = value - MARGIN; // Replaces the lowest value in list but cannot be zero, which would be connection error
            if (value > sensorMax[i]) sensorMax[i] = value + MARGIN; // Replaces the highest value in list for this sensor
        }
        delay(20); // Errors prevalent without buffer delay
    }
    delay(200); // debounce button

    digitalWrite(ledCalibrate, LOW);

    // Wait for button press to start running
    while (digitalRead(buttonPin) == LOW) {
        continue;
    }
    while (digitalRead(buttonPin) == HIGH) { // If button held down
        continue;
    }
    delay(200); // debounce
    robotState = RUNNING; // start moving (PID control)
    digitalWrite(ledRunning, HIGH); // Turn Green LED on to signal Movement control
}
```

Fig. 12. Calibration state function.

Once the button is pressed again, the robot briefly returns to idle mode so that it can be safely repositioned at the starting point of the course. The red LED turns off, and on the next button press, the robot enters its active line-following mode and switches on the green LED to confirm that control has begun. At this point the software begins a continuous control loop. Each raw 10-bit sensor reading is immediately mapped into a normalised scale, typically from 0 to 10, where 0 represents white and 10 represents black, using the previously recorded calibration values for that specific sensor. This per-sensor normalisation ensures that all sensors produce consistent and comparable output, even if their raw voltage ranges differ. See Fig. 13 below for code implementation of the read sensor pins function.

```

void readSensors() {
  for (uint8_t i = 0; i < 5; i++) {
    uint16_t value = analogRead(sensorPins[i]);
    uint8_t minV = ((sensorMin[i]));
    uint16_t maxV = ((sensorMax[i]));
    uint8_t value_mapped = map(value, minV, maxV, 10, 0); // dark line → higher value
    value_mapped = constrain(value_mapped, 0, 10);
    sensorValues[i] = value_mapped;
  }
}

```

Fig. 13. Read sensors and reverse map values function.

To determine the position of the line, the software scans the normalised sensor values from left to right and selects the first sensor whose output exceeds a chosen threshold value (for example 7 or higher). This sensor index directly represents the position of the black line relative to the centre of the robot and is immediately converted into an error value, such as -2 when the line is far left, 0 when centred, or +2 when far right. If no sensor currently detects the line, the software retains the last known line position rather than assuming it has disappeared entirely; this prevents erratic behaviour if the line briefly exits the sensor array during a sharp turn. See Fig. 14 below for the code implementation of line position calculation function.

```

// apply weight to value based on line position for given sensor
uint8_t calculateLinePosition() {
  for (uint8_t i = 0; i < 5; i++) {
    if (sensorValues[i] >= 7) {
      lastPosition = i;
      return lastPosition;
    }
  }
  return lastPosition; // remember corner
}

```

Fig. 14. Calculate line position function.

This error value is passed into a PID controller, which combines proportional, integral, and derivative terms to compute a correction value. The proportional term responds to the current position error, the integral term accumulates the error over time to eliminate steady-state bias, and the derivative term reacts to rapid changes in error to reduce overshoot. The resulting correction value is then applied to the motor speeds such that it is added to the base speed of one motor and subtracted from the other, causing the robot to steer smoothly back toward the line. Both motor speeds are constrained to remain within a safe range before being written to the motor driver. This control loop repeats continuously until the button is pressed once more, at which point the robot exits back to the idle state. See Fig. 15 below for the code implementation of the running PID control function.

```

// RUNNING STATE
void runningState() {
    delay(20); // make sensor readings nicer
    readSensors(); // read the sensors for calculation
    uint8_t position = calculateLinePosition(); // 0 to 4 (center = 2)
    int8_t error = 2 - position; // normalize error
    //integral += error; // integral for later
    int8_t derivative = (error - lastError);
    //lastError = error;
    int8_t correction = Kp * error + Kd * derivative; //Ki * integral
    int8_t leftSpeed = baseSpeed + correction;
    int8_t rightSpeed = baseSpeed - (correction);
    leftSpeed = constrain(leftSpeed, 0, maxSpeed); // the speeds must be between 0 and maxSpeed
    rightSpeed = constrain(rightSpeed, 0, maxSpeed);
    analogWrite(leftMotorPWM, leftSpeed); // move the motors
    analogWrite(rightMotorPWM, rightSpeed);
    // Stop if button pressed again
    if (digitalRead(buttonPin) == HIGH) {
        while (digitalRead(buttonPin) == HIGH) {
            continue; // holding the button
        }
        delay(200); // debounce
        robotState = IDLE; // shift state back to idle
        digitalWrite(ledRunning, LOW); // turn green LED off
    }
}

```

Fig. 15. Running state function.

Design Justification

Initially, the control strategy was intended to be more explicitly behaviour based. The goal was to make the robot recognise track features such as corners and respond to them using conditional logic. This approach was attractive as it would be able to deal better with several potential instances perfectly. However, it quickly became clear that implementing reliable corner detection using only reflectance sensors was far more complex than anticipated. Small variations in lighting, floor reflectivity, or sensor tolerance could easily miss the line or misfire completely. In addition, explicitly coding corner handling required extensive manual tuning and created rigid behaviour that did not generalise at all.

This led to a shift toward a continuous feedback approach using a PID wheel-speed controller. Rather than trying to understand the shape of the track, the robot simply corrects its position based on how far it currently is from the line. This method is not only mathematically superior, but also far more generalisable it works equally well for gradual curves and sudden transitions without requiring the robot to ‘know’ the track layout beforehand. Alternative software strategies such as bang-bang (on/off) control were considered but as the PID controlled motor speed was the next increment from the bang-bang, PID was favoured over it. The PID approach offered potential for smoother, self-correcting behaviour, especially when combined with the decision to preserve the last known line position when temporarily lost in software.

On the sensing side, one concept explored was spacing the two outermost sensors further from the centre, aiming to better capture wide-angle bends. However, simulations and physical tests showed that this increased the chance of accidentally detecting off-track reflections or stray lines, especially during repositioning, (in lost mode). The chosen design instead prioritised consistency, using a tightly grouped sensor array and relying on calibration and PID correction rather than physical overspreading. To handle hardware variation and unpredictable environmental conditions, a manual calibration phase was implemented in software dynamically capturing each sensor's minimum and maximum values under real lighting conditions before line following begins. This eliminated the need to manually hard-code thresholds and significantly improved robustness and adaptability to differing conditions.

In hardware selection, the ATmega328P-U TRT (through-hole) format was deliberately chosen over a surface-mount alternative. Although the SMD version would reduce programming error potentially. The through-hole part made hand-soldering, probing, and part replacement simple, one of the most critical advantages in a beginner prototyping environment. The MCU's internal ADC was selected over an external ADC module to reduce board and project complexity while still offering sufficient resolution to sense the line position at a given instant.

In summary, both hardware and software design decisions were made through trial and error of potential solutions that were either too difficult, too inflexible or weren't optimal for the approach that delivered on adaptability and simplicity. The final system is designed intentionally to be simple, tolerant of noise and lighting variation, and designed to be maintainable and educational.

Mechanical Design

Sensor placement, operating height, and spacing were all good and allowed for optimal readings under the conditions of the track. The spacing of the sensors prevented an absence in line readings when between the two furthestmost sensors. The small PCB also allowed for a non-obstructive placement on the body, with an elevated design being chosen as to allow maximum space for the other components. However, the length of the base plate was something that became an overlooked mistake as it created poor weight distribution across the LFR. The original CAD model for the assembled LFR can be seen below in Fig. 16:

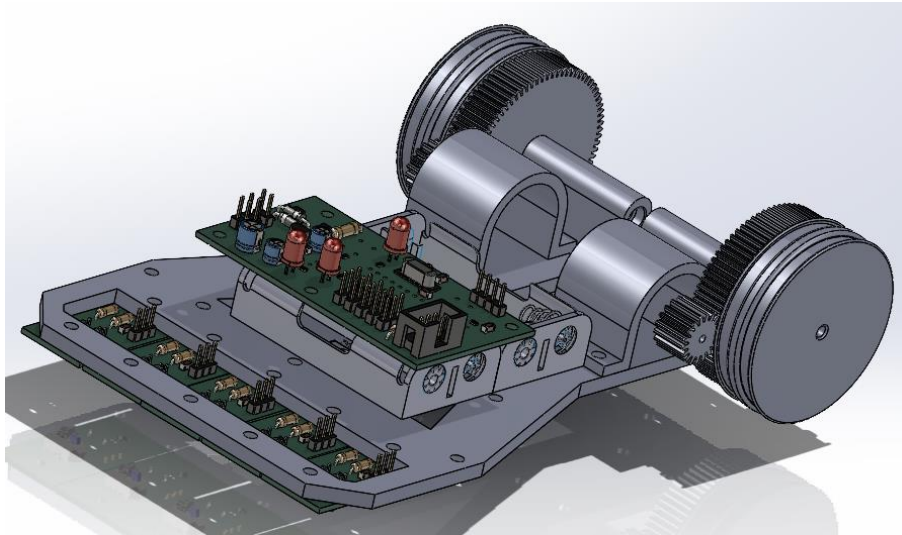


Fig. 16. CAD Model of LFR Assembly

Placement of weight is incredibly important for robots of this size; there is a lot of weight to move and the motors available are comparatively weak. While the use of gearing ratios (to gear down the motors and increase torque) did help, it was not enough to ensure consistent movement of the robot on the provided track. This will be outlined in further detail later in Iterative Design, but this was a consistent issue during testing caused by problems early in the projects design stage; looking at the original design for the LFR (Fig. 17):

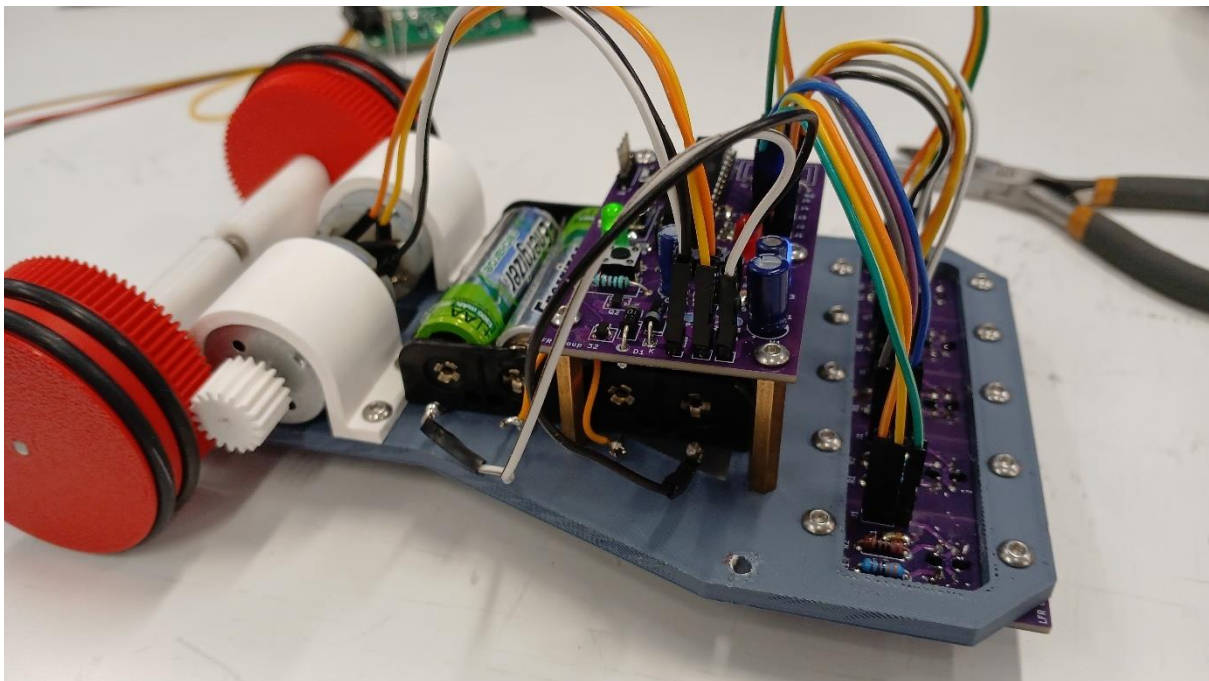


Fig. 17. Photo of Original Battery Placement

This design had the axle at the back, which works intuitively to maximise turning capabilities and makes balance easy, but leaves most of the weight of the sensors, PCB

and batteries to be pushing down on the front of the robot causing increased friction on the front wheel. To solve this, it was decided that the batteries had to be moved back onto the motors and rear axle to relieve most of that force, this new placement can be seen in Fig. 18:

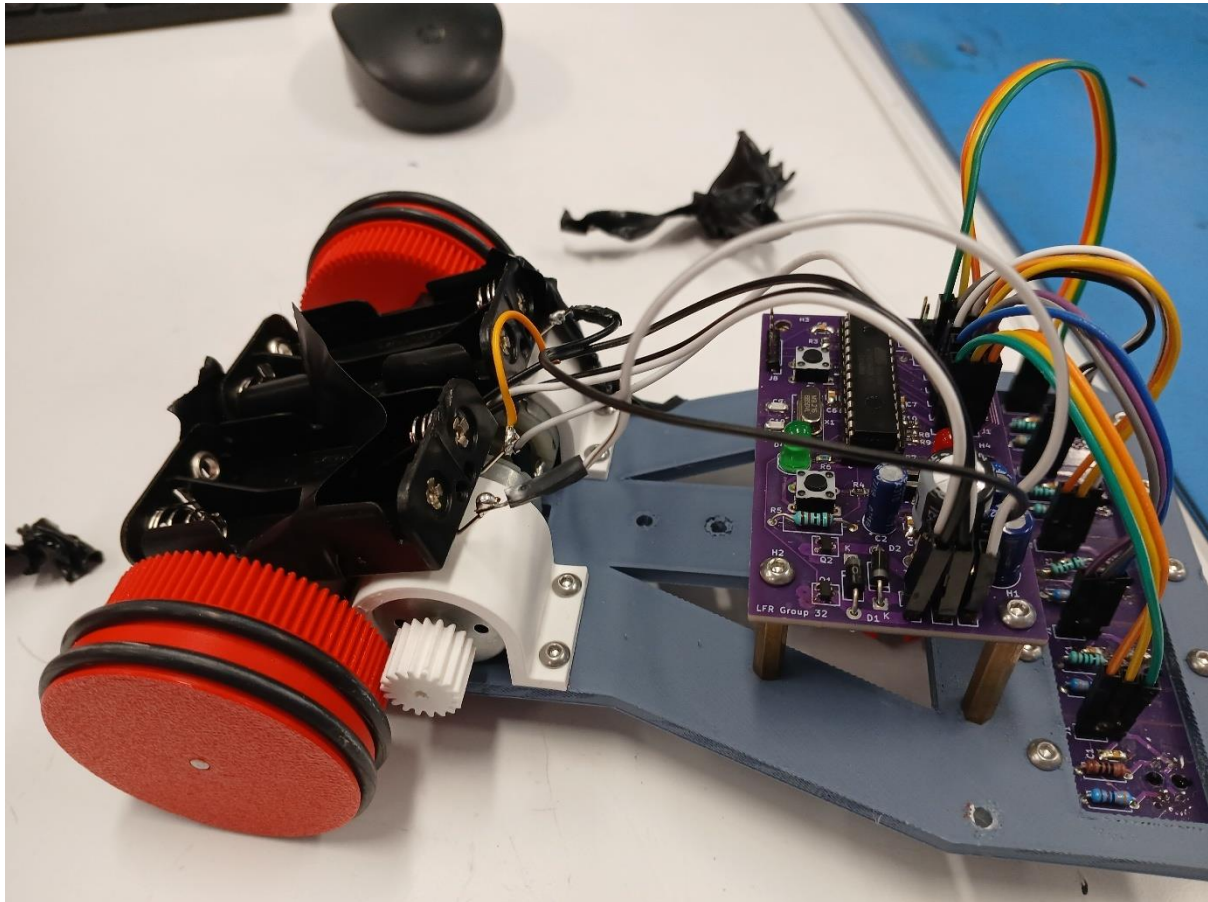


Fig. 18. Photo of New Battery Placement

Something that could have been greatly beneficial to the final design would have been to expand upon this idea, to further reduce the footprint of the LFR by stacking components vertically as was originally done with the raised PCB.

As touched on before this LFR design implemented a gear connection between the motors and the wheels, this had two main effects: Increasing torque which helped with moving the heavier weights and slowing down the total speed of the LFR. While going slower seems like a negative, due to the high rpm of the Multicomp MM28 motors we were using, the slower speed allowed for greater control of the LFR that would not have been possible without these gears, even with PWM control. Maximising the gear ratio was very important to the overall effectiveness of the design, and the implementation of a more complex 3-gear design could have greatly improved power and control. Unfortunately, as this design only has 2 gears the gear ratio was limited to the size of the wheels and the space in the back of the LFR, which limited the possible power of the design.

Iterative Design

A clear use of iterative design in the LFR was the front wheel, most online LFRs decide to opt out of using a traditional wheel, instead relying on ‘skid feet’: small smooth feet that provide little resistance and allow the LFR to slide along the ground. Below are the three first trialled feet concepts (Fig. 19):

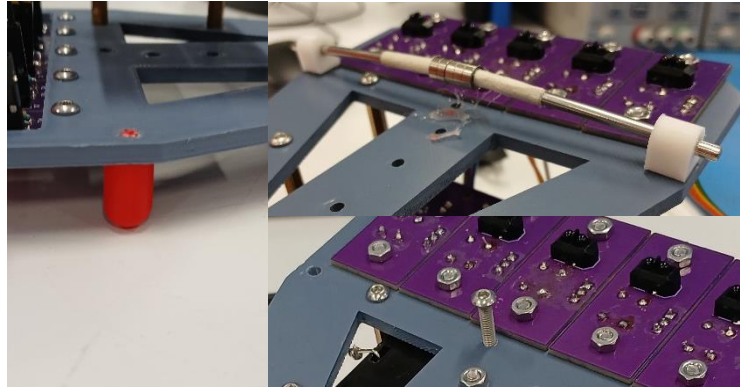


Fig. 19. Prototype Front Wheel Designs

Attempting to copy this design, prototype skid feet were designed and printed, but they created far too much friction both due to the imperfections in the 3D printed material and the distribution of weight in the LFR. An alternative bearing-based wheel was trialled, and while it had far less vertical friction, it had even more horizontal friction which prevented the LFR from turning. Another option that was tried was an extruding bolt, while the smooth metal did slide better than the printed option it was still not enough to get the LFR to move. It was clear a more mechanical solution was needed, and a castor wheel felt like the perfect fit. As seen in Fig. 20 a fully 3D printed ‘shopping trolley’-esque wheel was made, alongside a bearing wheel housing scaled to fit a marble:

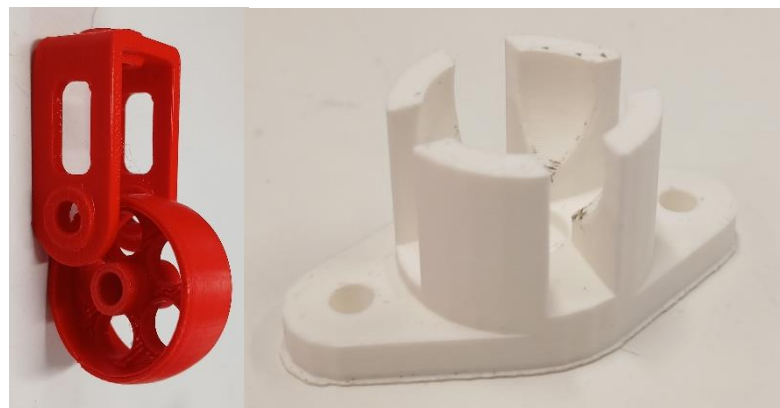


Fig. 20. Two Front Wheel Test Prints

While the shopping trolley wheel was impractical and large, the marble wheel had potential and with some minor modification was able to provide (mostly) frictionless motion in both the vertical and horizontal axes allowing for smooth and controlled motion in the LFR, full implementation of the marble wheel shown in Fig. 21:

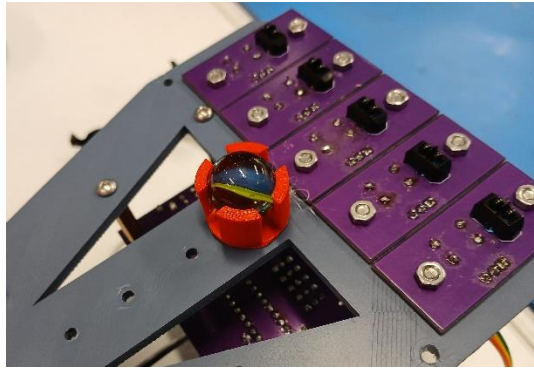


Fig. 21. Fully Assembled Marble Wheel

Final Design

Full Final Design shown in Fig. 22:

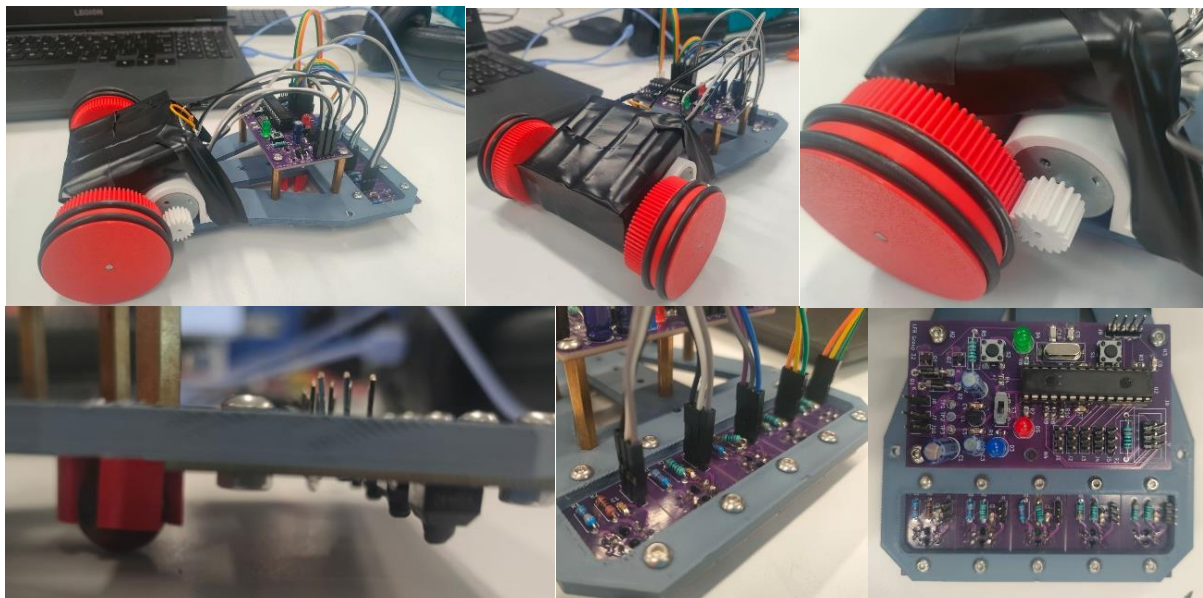


Fig. 22. Collage of Final Design and Key Features Photos

Key mechanical features of the final LFR design were the omnidirectional marble wheel, the geared connection between the motors and the back wheels, and the front bumper. The marble wheel and the geared connection were covered extensively above, but the front bumper was not; this bumper was an extension of the LFR body and both gave a connection point to the second mounting hole of the sensors PCBs (preventing unwanted movement in the sensors) and protected the sensor boards from damage during the several high speed collisions the LFR underwent during testing.

Results

Unfortunately, during the development of this LFR several serious issues prevented us from being able to ever get the LFR to complete a lap of the test course. The first issue was the poor weight distribution. This prevented the LFR from moving at all unless the motor values were high enough. However, this value would also result in the robot

travelling too fast to control. Several coding fixes were attempted, but as discussed earlier in this report the solution ended up being mechanical.

The second issue was far worse and eventually led to the failure of this project. The issue was a mysterious and sudden acceleration of the LFR, which occurred frequently and almost completely at random. Further testing showed these accelerations were not due to any software bugs but were instead physical issues with the circuitry between the board and motors. This issue was seemingly fixed following the implementation of a new 100 μ F capacitor between ground and VCC, during this relieve we were able to get the LFR fully running and proved that the sensors, PID control and motor speeds were all running as expected. While some PID values still needed to be tweaked the LFR was successfully able to complete the rounded square track. However, as more PID value testing was happening, the LFR suddenly shot off full speed into a wall and then became almost completely unusable. Because of this it appeared the main PCB was damaged as it could no longer driver one motor completely and the other was so weak even at max speed it would not even move, (it just wined). While a backup PCB had been prepared it seemed to have a soldering error about the MOSFET legs as it would not driver the motors at all when plugged in.

The sensors would work fine and could generate a good range of values that were sadly not used fully in this project. However, the graph below (Fig. 23) shows how the sensors were made with the correct spacing. This is because the transition state of each sensor coincides with another with the overlap located below the halfway point of all ranges.

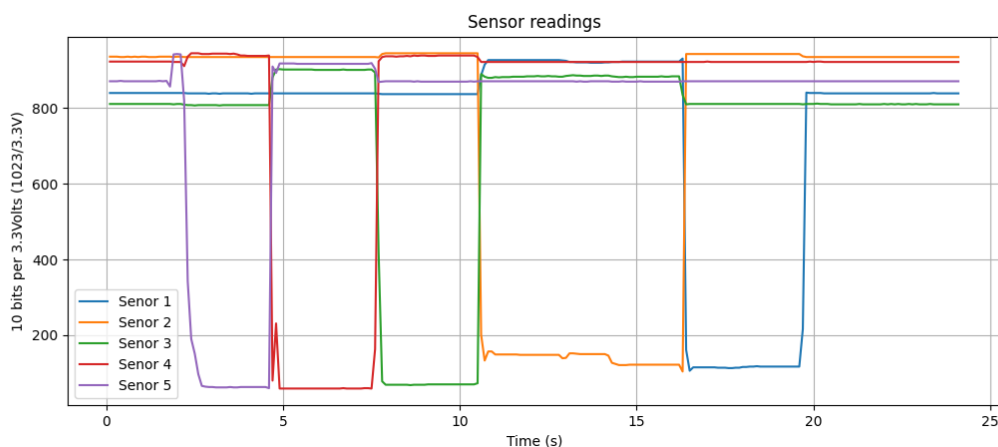


Fig. 23. Graph of all Sensor Readings over Time

This would mean that further programming and software development could result in a near perfect smoothing and sensor transition curve.

The program that was used in this case produces 0 - 10 output value for each sensor determined by darkness of sensors FOV. Using this to compare the analogue PWM values written to the motor. see this below (Fig. 24).

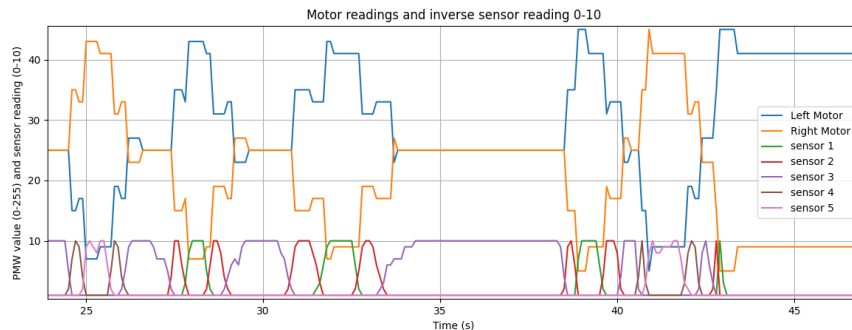


Fig. 24. Motor and inverse sensor readings

This figure displays a relatively good proportional speed response profile. However, it still does need more tuning time and software improvements to make the most of the analogue sensor values and to produce the desired smooth curve. These improvements were cut short, because of the hardware issues previously mentioned.

Conclusion

Throughout this project, it was a given requirement to learn about creating a mechatronics solution to a new and constrained problem. Specifically, this project taught a lot about designing PCBs for specific purposes, choosing components for the robot, and troubleshooting issues. Although the LFR didn't complete the course, it still managed to follow simple square track. Some parts of the project that worked well were the sensor software, which allowed the sensors to give consistent readings and the geared transmission to the wheels which successfully improved torque and lowered the wheel speed without significant slipping and efficiency loss. The major issue of this project (the sudden LFR acceleration) was never fully fixed and eventually fully prevented further testing, but despite this it was still possible to fix what could be fixed and make massive strides towards almost finishing this project. Something that could have been done better would have been to assign tasks better between team members, to clearer define the roles of each team member and better manage team resources.

Appendices

Appendix A

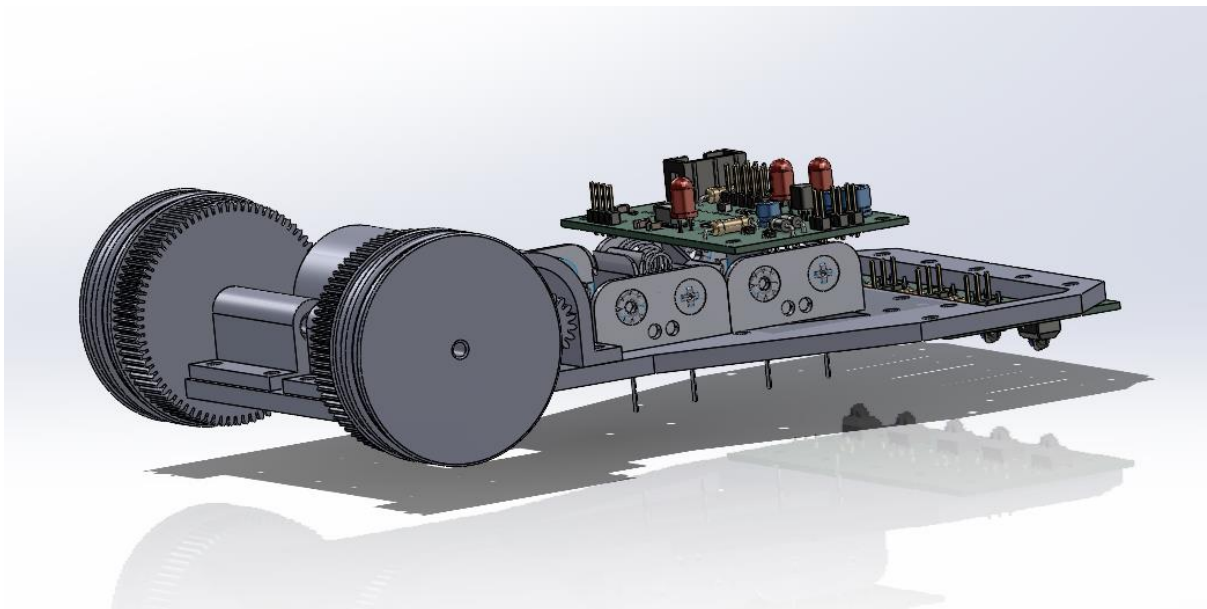
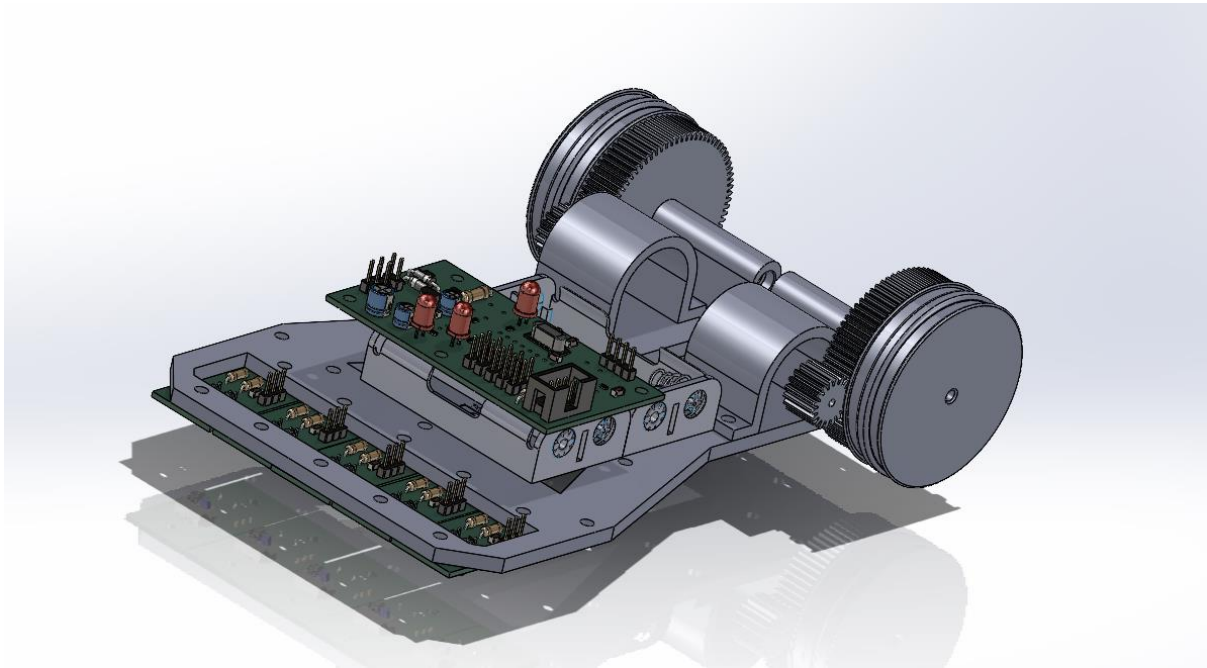
AI Disclosure Statement:

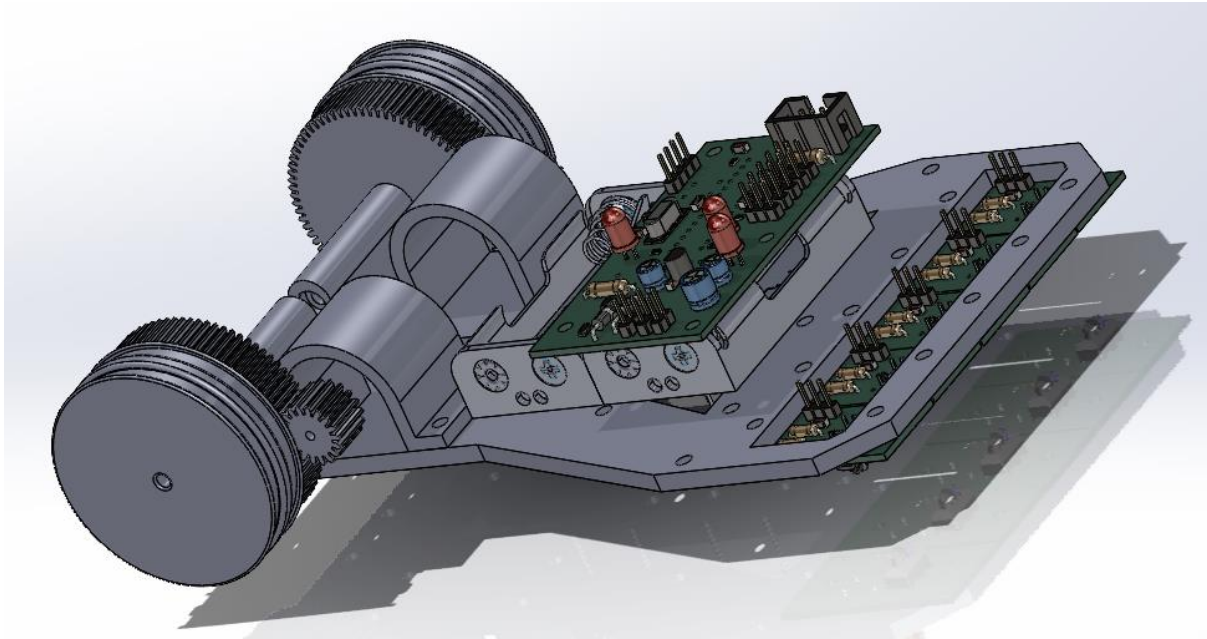
Generative AI tools such as ChatGPT and Copilot were used in the development of this project. All AI generated material was reviewed for errors. The primary

uses of AI were generating small sections of code for complex functions, debugging, spelling and grammar checking, Report structuring, PCB design inquiries and other trouble shooting.

Appendix B

CAD models of original LFR design:





Appendix C

All code implemented in working LFR:

```
/ Line Following Robot (PID)

// 5x TCRT5000 sensors (analog inputs)
// 2x PWM motor outputs
// 1x Active HIGH button input
// 2x LED outputs for state indication

// Pin Configuration
const uint8_t sensorPins[5] = {A0, A1, A2, A3, A4};
const uint8_t leftMotorPWM = 6;
const uint8_t rightMotorPWM = 5;
const uint8_t buttonPin = 7;
const uint8_t ledCalibrate = 9;
const uint8_t ledRunning = 8;

uint8_t MARGIN = 5; //could be higher for various tracks

// PID Constants
uint8_t Kp = 8;
//uint8_t Ki = 0;    // tune later if needed
uint8_t Kd = 2;

// Variables
uint8_t sensorValues[5];
uint16_t sensorMin[5];
uint16_t sensorMax[5];
```

```

// Globals needed for cross function operation (some could be refined out of
global memory)
int8_t lastError = 0;
uint8_t lastPosition = 2;
uint8_t ticks = 5;
//int16_t integral = 0;
uint16_t time_step = 0;

// Robot speed limits
const uint8_t baseSpeed = 25;    // adjust motor strength
const uint8_t maxSpeed = 65;

// State management
enum State {
    IDLE,
    CALIBRATING,
    RUNNING
};
State robotState = IDLE;

// Setup
void setup() {
    Serial.begin(9600);

    pinMode(leftMotorPWM, OUTPUT);
    pinMode(rightMotorPWM, OUTPUT);
    pinMode(buttonPin, INPUT);
    pinMode(ledCalibrate, OUTPUT);
    pinMode(ledRunning, OUTPUT);

    // Initialize sensor calibration ranges
    for (int i = 0; i < 5; i++) {
        sensorMin[i] = 1023;
        sensorMax[i] = 0;
    }

    analogWrite(leftMotorPWM, 0);
    analogWrite(rightMotorPWM, 0);
}

// Loop
void loop() {
    switch (robotState) {
        case IDLE:
            idleState();

```

```

        break;
    case CALIBRATING:
        calibrateState();
        break;
    case RUNNING:
        runningState();
        break;
    }
}

// IDLE STATE
void idleState() {
    digitalWrite(ledCalibrate, LOW); // both LED's off
    digitalWrite(ledRunning, LOW);
    analogWrite(leftMotorPWM, 0); // set motors to off
    analogWrite(rightMotorPWM, 0);

    // Wait for button press to start calibration
    if (digitalRead(buttonPin) == HIGH) {
        while (digitalRead(buttonPin) == HIGH) {
            continue; // if button is held down
        }
        delay(200); // debounce
        robotState = CALIBRATING; // shift to calibration state
    }
}

void calibrateState() {
    digitalWrite(ledCalibrate, HIGH); // Red LED on meaning calibration move
    active
    while (digitalRead(buttonPin) == LOW) { // Calibrate till button press
        for (uint8_t i = 0; i < 5; i++) {
            int16_t value = analogRead(sensorPins[i]); // Read 10 bit sensor value
            if ((value < sensorMin[i]) && (value != 0)) sensorMin[i] = value -
MARGIN; // Replaces the lowest value in list but cannot be zero, which would
be connection error
            if (value > sensorMax[i]) sensorMax[i] = value + MARGIN; // Replaces the
highest value in list for this sensor
        }
        delay(100); // Errors prevalent without buffer delay
    }
    delay(200); // debounce button
    digitalWrite(ledCalibrate, LOW);
    // Wait for button press to start running
    while (digitalRead(buttonPin) == LOW) {
        continue;
    }
}

```



```

while (digitalRead(buttonPin) == HIGH) { // if button held down
    continue;
}
delay(200); // debounce
robotState = RUNNING; // start moving (PID control)
digitalWrite(ledRunning, HIGH); // Turn Green LED on to signal Movement
control
}

// RUNNING STATE
void runningState() {
    delay(100); // make sensor readings nicer
    readSensors(); // read the sensors for calculation
    uint8_t position = calculateLinePosition(); // 0 to 4 (center = 2)
    int8_t error = 2 - position; // normalize error
    //integral += error; // integral for later
    int8_t derivative = (error - lastError);
    ticks++;
    if (ticks > 5) {
        lastError = error;
        ticks = 0;
    }

    int8_t correction = Kp * error + Kd * derivative; //Ki * integral
    int8_t leftSpeed = baseSpeed + correction;
    int8_t rightSpeed = baseSpeed - (correction);
    leftSpeed = constrain(leftSpeed, 0, maxSpeed); // the speeds must be between
0 and maxSpeed
    rightSpeed = constrain(rightSpeed, 0, maxSpeed);
    // Stop if button pressed
    // For display of figures through serial port communications (remove for
operation)
    time_step++;
    Serial.printf("%d, %d, %d", time_step, leftSpeed, rightSpeed);
    for (int i = 0; i < 5; i++) {
        Serial.printf(", %d", sensorValues[i]);
    }
    Serial.println();
    Serial.flush();
    // drive motors
    analogWrite(leftMotorPWM, leftSpeed);
    analogWrite(rightMotorPWM, rightSpeed);

    if (digitalRead(buttonPin) == HIGH) {
        while (digitalRead(buttonPin) == HIGH) {
            continue; // holding the button
        }
    }
}

```

```

    delay(200); // debounce
    robotState = IDLE; // shift state back to idle
    digitalWrite(ledRunning, LOW); // turn green LED off
}
}

// Read sensors (normalized 0-10)
void readSensors() {
    for (uint8_t i = 0; i < 5; i++) {
        uint16_t value = analogRead(sensorPins[i]);
        uint8_t minV = ((sensorMin[i]));
        uint16_t maxV = ((sensorMax[i]));
        uint8_t value_mapped = map(value, minV, maxV, 10, 0); // dark line →
higher value
        value_mapped = constrain(value_mapped, 0, 10);
        sensorValues[i] = value_mapped;
    }
}

// apply weight to value based on position
uint8_t calculatelinePosition() {
    for (uint8_t i = 0; i < 5; i++) {
        if (sensorValues[i] >= 7) {
            lastPosition = i;
            return lastPosition;
        }
    }
    return lastPosition; // remember corner
}

```

Appendix D

More figures of plotted data from sensor readings and motor values:

